

# HW2

## 李毅PB22051031

### 1. Which of the following instructions should be privileged?

- a. Set value of timer.
- b. Read the clock.
- c. Clear memory.
- d. Issue a trap instruction.
- e. Turn off interrupts.
- f. Modify entries in device-status table.
- g. Switch from user to kernel mode.
- h. Access I/O device.

a, c, e, f, g, h.

### 2. 在哪些情况下，我们使用的个人笔记本电脑运行在管理/特权/内核模式？如果用户程序要主动触发进入操作系统内核中执行，据你了解可以如何进行？（提供一种方法即可）

- 系统调用（System Call）：系统调用是最常见的用户态切换到内核态的方式。用户程序通过系统调用请求操作系统执行特定的任务，例如文件操作、网络通信或进程管理。
- 中断（Interrupt）：中断是 CPU 从用户态切换到内核态的另一个重要机制。中断可以是硬件中断或软件中断。
- 异常（Exception）：异常是 CPU 在执行指令过程中检测到问题时触发的一种机制。它包括以下几种类型：陷阱（Trap）：有意触发的异常，例如调试器设置的断点。故障（Fault）：可以被修复的问题，例如页错误（Page Fault）。终止（Abort）：不可恢复的严重错误，例如硬件故障。
- 特权指令（Privileged Instruction）：某些指令只能在内核态执行，例如直接访问硬件设备、修改中断表或管理内存分页。如果用户态代码尝试执行这些指令，CPU 会触发异常并切换到内核态。
- 系统定时器中断：操作系统使用定时器中断实现多任务调度。定时器中断是由硬件定时器触发的，通常每隔固定时间（如 1 毫秒）触发一次中断，让操作系统切换到内核态进行任务切换。

用户程序无法直接执行内核代码，但可以通过系统调用（System Call）主动触发模式切换。这是唯一合法且安全的途径。

### 3. 用户接口有哪几种？OS提供服务最基本的方式是什么？什么是系统调用号？系统调用参数的传递方法常见的有哪三种？

(1) 操作系统提供三种主要用户接口：

- **命令行界面（CLLI）**：通过文本命令与系统交互（如Linux的Bash、Windows的CMD）。
- **图形用户界面（GUI）**：通过窗口、图标、按钮等视觉元素操作（如Windows桌面、macOS）。
- **应用程序接口（API）**：供程序员通过函数调用使用系统服务（如文件读写API、网络通信API）。

(2) 操作系统通过系统调用（System Call）提供服务

(3) 每个系统调用被分配一个唯一的系统调用号，用于内核快速定位对应的处理函数。

(4) 系统调用参数传递的三种常见方法：寄存器直接传递，栈传递，内存块指针传递

### 4. 常见的IPC机制包括哪5种？

- 管道（Pipe）
- 消息队列（Message Queue）
- 共享内存（Shared Memory）
- 信号量（Semaphore）

- 套接字 (Socket)

## 5. OS特征有哪些？其中，最基本的特征是什么？并发和并行的关系是什么？异步性具体指什么？

(1) 操作系统的四个特征分别是：并发，共享，虚拟，异步。并发和共享是两个最基本的特征，二者互为存在条件。

(2)

- 并发 (Concurrency)：多个任务在同一时间段内运行，但并不一定同时执行，可能是交替进行的（时间片轮转）。适用于单核或多核处理器。
- 并行 (Parallelism)：多个任务真正同时执行，通常发生在多核处理器或多机集群环境中。
- 关系：并行是一种特殊的并发，所有并行都是并发，但并发不一定是并行。并发强调任务的交替执行，而并行强调多个任务的同时执行。

(3) 异步性指程序的执行流程不按照固定的线性顺序进行，而是由事件驱动，某些任务在等待资源时不会阻塞，而是继续执行其他任务，等到资源可用时再继续执行之前的任务。

## 6. 请列举7种不同的OS体系结构？根据你的理解，Linux操作系统采用了什么样的体系结构？

- 单体 (Monolithic) 结构
- 微内核 (Microkernel) 结构
- 混合内核 (Hybrid Kernel) 结构
- 层次 (Layered) 结构
- 客户端-服务器 (Client-Server) 结构
- 外核 (Exokernel) 结构
- 虚拟机 (Virtual Machine) 结构

Linux 主要采用单体 (Monolithic) 内核结构，但引入了模块化 (Modular) 机制，可以动态加载/卸载内核模块，使其具备一定的可扩展性和灵活性。因此，Linux 既具有单体内核的高效性，又吸收了微内核的一些优势，如模块化驱动管理。

## 7. 程序并发执行的特征和条件分别是什么？

(1) 程序并发执行的特征：

- 间断性：由于多个程序共享CPU，操作系统采用时间片轮转或调度策略，导致程序的执行是断续进行的，而不是连续的。
- 共享性：并发程序共享系统资源（如CPU、内存、I/O设备等），可能会产生竞争（如多个进程同时访问同一文件）。
- 独立性：每个并发程序都有自己的执行流程，逻辑上是独立的，但可能需要协作完成某些任务。
- 不确定性：由于调度策略、资源分配、同步机制等因素，程序的执行顺序可能不固定，导致结果可能不同。
- 交互性：并发进程/线程之间可能需要通信与协作，例如进程间通信 (IPC)、同步（如互斥锁、信号量）。

(2) 程序并发执行的条件：

- 可分性 (Decomposability)：任务可以拆分成多个独立或相互协作的子任务，每个子任务可以独立执行。
- 多道程序环境 (Multiprogramming Environment)：计算机支持多个程序同时驻留在内存中，以便调度和执行。
- CPU调度机制：操作系统需要具备调度能力，使多个进程能够交替运行，例如时间片轮转、优先级调度等。

- 资源共享机制：系统允许多个进程共享资源，并提供访问控制，如文件系统、I/O设备、内存共享等。
- 同步与互斥机制：需要合适的同步（如信号量、条件变量）和互斥（如锁）机制，以避免竞争条件（Race Condition）和死锁（Deadlock）。

## 8. 进程的5大特征是什么？其中，最基本的特征是哪个，其具体含义是什么？进程的独立性具体指什么？

(1) 进程的5大特征：

- 动态性 (Dynamic)：进程是程序的执行过程，会随着运行状态的变化而动态创建、调度、终止。
- 并发性 (Concurrency)：多个进程可以在同一时间段内运行，并发共享CPU等系统资源。
- 独立性 (Independence)：每个进程都有独立的地址空间、数据和运行环境，彼此相互隔离。
- 异步性 (Asynchronism)：进程的执行受调度策略和资源分配的影响，执行速度不可预测。
- 结构性 (Structure)：进程由程序段、数据段和PCB（进程控制块）组成，PCB记录进程状态、资源信息等。

(2) 最基本的特征：动态性，含义：

- 进程是程序在计算机系统中的执行实体，不是静态的，而是随着运行不断变化的。
- 进程的整个生命周期包括创建、就绪、运行、阻塞和终止。
- 进程的动态性决定了操作系统的调度管理以及资源分配方式。

(3) 进程的独立性指进程在逻辑上相互独立，每个进程都有独立的地址空间，不会直接访问其他进程的内存数据。进程的执行依赖于自身的程序代码和数据，不会直接影响其他进程。

## 9. 谈谈你对进程地址空间中堆和栈的理解。

(1) 堆 (Heap)：用于动态内存分配，从低地址向高地址扩展。适用于需要动态分配的场景，如动态数据结构（链表、树等）。分配和释放速度相对较慢，可能导致内存碎片问题。

(2) 栈 (Stack)：用于函数调用、局部变量分配，从高地址向低地址增长。分配和释放速度快，无需手动管理内存。但不能灵活调整大小，局部变量的生命周期受限于函数作用域

## 10. 调研：请结合某一版本的Linux内核源代码，找到task\_struct中与进程状态相关的数据项，并说明Linux中的进程有哪些状态，分别代表什么含义？

在Linux内核源码（如Linux 5.x）中，`task_struct` 结构体定义在 `include/linux/sched.h` 文件中，其中与进程状态相关的关键字段为：

```
1 | volatile long state;
```

该字段用于表示进程的当前状态。状态值使用了一系列宏定义，可在 `include/linux/sched.h` 头文件中找到，例如：

```
1 | /*
2 |  * Task state bitmask. NOTE! These bits are also
3 |  * encoded in fs/proc/array.c: get_task_state().
4 |  *
5 |  * We have two separate sets of flags: task->state
6 |  * is about runnability, while task->exit_state are
7 |  * about the task exiting. Confusing, but this way
8 |  * modifying one set can't modify the other one by
9 |  * mistake.
10 | */
11 | #define TASK_RUNNING 0
12 | #define TASK_INTERRUPTIBLE 1
```

```
13 #define TASK_UNINTERRUPTIBLE 2
14 #define __TASK_STOPPED 4
15 #define __TASK_TRACED 8
16 /* in tsk->exit_state */
17 #define EXIT_DEAD 16
18 #define EXIT_ZOMBIE 32
19 #define EXIT_TRACE (EXIT_ZOMBIE | EXIT_DEAD)
20 /* in tsk->state again */
21 #define TASK_DEAD 64
22 #define TASK_WAKEKILL 128 /** wake on signals that are deadly **/
23 #define TASK_WAKING 256
24 #define TASK_PARKED 512
25 #define TASK_NOLOAD 1024
26 #define TASK_STATE_MAX 2048
27 /* Convenience macros for the sake of set_task_state */
28 #define TASK_KILLABLE (TASK_WAKEKILL | TASK_UNINTERRUPTIBLE)
29 #define TASK_STOPPED (TASK_WAKEKILL | __TASK_STOPPED)
30 #define TASK_TRACED (TASK_WAKEKILL | __TASK_TRACED)
```

---

## 2. Linux 进程的状态及其含义

在 Linux 内核中，进程状态通常分为以下几类：

| 状态宏                               | 值    | 含义                                                                                                        |
|-----------------------------------|------|-----------------------------------------------------------------------------------------------------------|
| <code>TASK_RUNNING</code>         | 0    | <b>运行状态</b> : 进程正在执行或准备执行, 可能正在 CPU 上运行, 也可能在就绪队列等待调度。                                                    |
| <code>TASK_INTERRUPTIBLE</code>   | 1    | <b>可中断睡眠状态</b> : 进程正在等待某个事件 (如 I/O 完成), 但可以被信号中断唤醒。                                                       |
| <code>TASK_UNINTERRUPTIBLE</code> | 2    | <b>不可中断睡眠状态</b> : 进程等待某个事件 (如磁盘 I/O), 但不会响应信号, 通常用于避免被中断的关键任务。                                            |
| <code>__TASK_STOPPED</code>       | 4    | <b>停止状态</b> : 进程暂停执行, 如接收到 <code>SIGSTOP</code> 、 <code>SIGTSTP</code> 信号, 等待 <code>SIGCONT</code> 继续执行。  |
| <code>__TASK_TRACED</code>        | 8    | <b>被跟踪状态</b> : 进程被 <code>ptrace</code> 调试 (如 <code>gdb</code> 调试), 处于暂停状态。                                |
| <code>EXIT_DEAD</code>            | 16   | <b>即将销毁状态</b> : 进程的 <code>task_struct</code> 结构即将被释放, 通常是 <code>EXIT_ZOMBIE</code> 状态的过渡状态。               |
| <code>EXIT_ZOMBIE</code>          | 32   | <b>僵尸状态</b> : 进程已终止, 但父进程尚未回收其退出状态信息 ( <code>wait()</code> )。进程仍占用 <code>task_struct</code> 结构, 但不占用其他资源。 |
| <code>TASK_DEAD</code>            | 64   | <b>死亡状态</b> : 进程即将结束, 等待内核清理。                                                                             |
| <code>TASK_WAKEKILL</code>        | 128  | <b>可被唤醒但带有终止信号</b> : 当某些杀死进程的信号到来时, 该进程会立即被唤醒并终止。                                                         |
| <code>TASK_WAKING</code>          | 256  | <b>正在唤醒状态</b> : 进程处于被唤醒的过程中, 正在从等待队列移动到就绪队列。                                                              |
| <code>TASK_PARKED</code>          | 512  | <b>暂停但可被再次执行</b> : 通常用于调度器优化, 表示进程当前未准备好运行, 但稍后可以恢复。                                                      |
| <code>TASK_NEW</code>             | 2048 | <b>新创建状态</b> : 表示进程刚创建, 尚未被调度执行。                                                                          |

## 参考文献:

- 【1】[CPU 用户态切换到内核态的触发机制详解-腾讯云开发者社区-腾讯云](#)
- 【2】[进程间通信（IPC）方法总结\(一\)-12oz-博客园](#)
- 【3】[操作系统的四大特征-知乎](#)
- 【4】王道操作系统考研复习指导2024
- 【5】[挑战408——操作系统（3）——程序的并发执行-知乎](#)
- 【6】[Linux进程描述符task\\_struct结构体详解-知乎](#)